

Laboratorio di Ingegneria Informatica – Esonero 1:

Pipeline di Parsing Web per il Chatbot Minerva

Matteo Martin Console

Matricola: 2112194

console.2112194@studenti.uniroma1.it

Alice Agneni

Matricola: 2128893

agneni.2128893@studenti.uniroma1.it

Giulia Codella

Matricola: 2118019

codella.2118019@studenti.uniroma1.it

1 Obiettivo del Progetto

Il progetto realizza una pipeline end-to-end per l'acquisizione e l'analisi di documenti provenienti da sorgenti web eterogenee, a supporto dell'LLM nazionale **Minerva** (Sapienza NLP / Babelscape). Il sistema, dato un URL appartenente a uno dei domini supportati, scarica la pagina, ne estrae il testo informativo pulito in formato Markdown eliminando il rumore tipico delle pagine web (header, footer, banner pubblicitari, elementi di navigazione), valuta la qualità del testo estratto confrontandolo con un Gold Standard costruito manualmente, ed espone l'intera funzionalità attraverso una REST API e un'interfaccia Web, il tutto coordinato in un'architettura multi-container Docker Compose.

2 Organizzazione del Codice

Il progetto segue la struttura richiesta dalla specifica e si struttura in due microservizi separati (backend e frontend), ciascuno con il proprio Dockerfile e requirements.txt, coordinati da un unico docker-compose.yaml che monta come volumi condivisi la directory gs_data/ (Gold Standard in formato JSON) e il file domains.json.

Backend (backend/src/server.py). Implementato con **FastAPI** (Python 3.10, porta 8003) e validazione I/O tramite **Pydantic**. Espone i seguenti endpoint della specifica: GET /parse (parsing da URL), POST /parse (parsing da HTML grezzo), GET /domains, GET /gold_standard, GET /full_gold_standard, POST /evaluate e GET /full_gs_eval. La funzione interna get_parsed_data agisce da smistatore: ispeziona il netloc dell'URL e instrada la richiesta al parser corretto. Tutti gli errori rilevanti (dominio non supportato, URL irraggiungibile, parser mancante) sono sollevati come HTTPException con i codici di stato appropriati (400, 404, 500).

La normalizzazione del Markdown prima del calcolo delle metriche avviene tramite la funzione remove_markdown, che converte il Markdown in HTML con mistune, rimuove i tag con BeautifulSoup, e collassa gli spazi multipli, seguendo esattamente la funzione di riferimento fornita dalla specifica.

Parser dedicati – backend/src/logic/. Sono implementati quattro parser asincroni, uno per ciascun dominio assegnato. Tutti usano **Crawl4AI** con BrowserConfig headless (Chromium via Playwright) e supportano sia il crawling diretto da URL sia il parsing di HTML grezzo passato tramite POST /parse: in quest'ultimo caso, l'HTML viene scritto in un file temporaneo e passato al crawler con il prefisso file://.

- **parser_wikipedia.py:** selettore CSS #mw-content-text per isolare il solo corpo dell'articolo; uno script JavaScript iniettato rimuove prima del crawling i tag sup, nav, footer, le classi .infobox, .navbox, .reflist, .mw-editsection e l'intera sezione "See also", garantendo un Markdown privo di note a piè di pagina e menu di navigazione.
- **parser_grammy.py:** selettore CSS main con esclusione esplicita dei tag header, footer, nav, aside, script, style e form. Il titolo viene estratto dai metadati del risultato; se assente o generico, viene ricavato dal primo heading Markdown con una regex.
- **parser_huddle.py:** selettore CSS article con esclusione di nav, aside, footer, script e style. Il Markdown grezzo viene ulteriormente depurato da una funzione clean_huddle_markdown che: rimuove i link mantenendo il testo, filtra le righe tramite una blacklist lessicale ("pubblicità", "facebook",

“twitter”, “adsbygoogle”) e scarta le righe più brevi di 30 caratteri che non siano heading.

- **parser_academia.py:** usa un User-Agent browser reale e un delay_before_return_html di 5 secondi per aggirare le protezioni anti-bot di Academia.edu. Adotta una strategia di estrazione a cascata: tenta prima di isolare il div con classe abstract/summary tramite regex sull’HTML grezzo; se il testo estratto è insufficiente (meno di 50 caratteri), ricade sul Markdown di Crawl4AI filtrato da clean_academia_markdown; come ultima risorsa, usa il meta tag og:description. Il titolo è ricavato in priorità dall’Open Graph tag, poi da <title>, poi da <h1>.

Metriche (backend/src/logic/metrics.py).

La funzione compute_token_level_eval implementa la token_level_eval obbligatoria: i token sono estratti tramite la regex [a-z0-9]+ applicata al testo in minuscolo, costituendo due insiemi (tokens_estratti e tokens_gs). Precision, Recall e F1 sono calcolati sull’intersezione degli insiemi, con gestione degli edge case (insiemi vuoti).

Gestione Gold Standard

(backend/src/logic/gs_manager.py). Carica i file JSON da gs_data/, indicizzati per dominio. Fornisce funzioni per cercare un singolo URL nel GS e per caricare l’intero GS di un dominio.

Frontend (frontend/src/frontend.py).

Server FastAPI (porta 8004) con rendering Jinja2. La pagina principale mostra un campo testo libero e un menu a tendina con tutti gli URL del Gold Standard. Dopo il parsing, visualizza affiancati l’HTML grezzo e il testo parsato; se l’URL è nel GS, mostra anche il confronto con il *gold text* e le metriche.

Infrastruttura. docker-compose.yaml avvia i due container con path relativi; i volumi gs_data e domains.json sono montati nel container backend.

3 Valutazione dei Parser

Per ciascuno dei quattro domini assegnati sono stati costruiti manualmente 5 Gold Standard (URL rappresentativi, testo informativo privo di elementi di navigazione). La valutazione automatica avviene

tramite GET /full_gs_eval, che esegue il parsing di ogni entry del GS in tempo reale e aggrega le metriche come media aritmetica. La tabella 1 riassume i risultati ottenuti.

Dominio	Prec.	Recall	F1
en.wikipedia.org	0.9952	0.9996	0.9974
grammy.com	0.9954	0.9937	0.9945
www.academia.edu	0.9866	1.000	0.9932
www.huddle.org	1.000	0.9997	0.9998

Table 1: Risultati medi della token_level_eval sui Gold Standard.

Tutti i parser si collocano nella fascia “Buono” (F1 > 0.80 secondo la specifica). Wikipedia ottiene i risultati migliori grazie alla struttura DOM regolare e al filtraggio JavaScript pre-crawling. Academia.edu, penalizzato dal paywall parziale e dalla struttura fortemente dinamica, raggiunge comunque la soglia minima grazie alla strategia di estrazione a cascata. Per huddle.org e grammy.com, domini con layout più variabile, le blacklist lessicali post-parsing si rivelano efficaci nel ridurre il rumore residuo